MURRAY STATE UNIVERSITY

Zero to Unbeatable

15-MINUTE DEMO · 5-MINUTE SECOND ACT · HOW MACHINES LEARN

You need: one phone, tablet or laptop. Open `index.html`. No install, no login, no network. Projecting? Settings → **Presenter mode**. **Do not train before the session**, and do not refresh mid-demo.

SETUP (60 SECONDS)

Open the file. Turn on **Presenter mode** if projecting. To reproduce a rehearsed run exactly, type your seed into **Rehearsal seed** and press **Reset to Era 0**. Leave the burst on **5,000** — the arc is tuned for it and takes three to five presses.

THE SCRIPT

0:00 Hand it to a student. "Beat it."
Era 0 has never played a game. It loses fast and stupidly — it will ignore a row you have almost finished. Let two or three people beat it. Let the room laugh at it.

> *"This has played zero games. It does not know what winning is. It knows how to put a mark in an empty square, and that is all."*

2:00 Press "Show its brain".
Every square reads `+0.00`, seen 0 times.

> *"That is the entire mind of your opponent. Nine numbers, all zero. No strategy, no line of code that says 'block the row'. When every number ties it flips a coin — which is exactly what you just watched."*

3:00 Press "Train the AI". Say nothing for three seconds.
Let them watch the counter blur past five thousand games. Point at the two live readings: **ε** falling — how often it still tries a random square — and its **win rate against random play** climbing. Then read the card: it now takes a free win and blocks your row.

4:30 Play Era 1. Two or three games.
It blocks now. It is annoying. You can still beat it — about one game in six — and when somebody does, that is your line:

> *"Five thousand games of experience and you still got it. So what is it missing?"*

6:30 Train again. Play Era 2.
This is the turn. Wins dry up, draws start, somebody loses and is annoyed about it. Let that land before moving on — the frustration *is* the lesson.

9:00 Train twice more. Play the newest era.
Between the third and fifth press the purple banner appears. Every game is a draw now. Invite the best player in the room to try. They will draw.

11:30 The beat that makes it stick: go back.
Choose **Era 0** from the dropdown and stomp it again. Then switch to the newest era and draw again. Same program, same code, different numbers.

> *"Nobody rewrote this. The only thing that changed is a list of numbers, and the only thing that changed the numbers was playing games and being told won, lost or drew."*

13:00 Read the banner aloud, then close on the bridge.
Tic-tac-toe has about five and a half thousand positions, so a table holds all of them. Chess and language cannot be listed at all — which is the whole reason big AI uses a neural network instead.

> *"Different brain. Same idea: play, get told how it went, adjust, repeat."*

DISCUSSION QUESTIONS

1. It played 5,000 games in three seconds. How many had you played before you stopped losing at this? What does that comparison tell us — and what does it hide?
2. Why does it deliberately play a bad square during training? What breaks if it always plays the move it currently rates best?
3. Forty per cent of its training is against an opponent playing at random. Why not train it only against itself, which is far stronger?
4. The last thing it learns is that answering two opposite corners in a corner loses. Why would that arrive after "block the row"?
5. Once trained, every opening square scores the same. Has it failed to learn something, or learned something?
6. It can never lose now. Is it *good at* tic-tac-toe, or has it memorised tic-tac-toe? Is there a difference, and does the difference matter?

MISCONCEPTIONS TO DRAW OUT

"It was secretly programmed with strategy and just pretends to start dumb."
Meet this head on. Show the brain at Era 0 — every value exactly zero. Then run **Settings → Run the full self-test** in front of them: it chi-squared tests 9,000 moves by a newborn and reports it indistinguishable from a coin flip. The code also measures that a newborn takes a free win no more often than chance. A hidden rule would show in both.

"It understands tic-tac-toe now."
It has a number attached to each of about five thousand board pictures. It cannot say what a row is, cannot play on a 4×4 board, cannot explain one choice. Ask the room what "understands" would have to mean and whether anything on screen meets it. Best argument you can have in the last two minutes.

"More training always keeps making it visibly better."
Watch the card across bursts. The first changes almost everything; by the third the landmark positions stop moving at all and only "positions it would now play differently" keeps falling. Improvement goes invisible, then stops — it has hit the ceiling of the game, and nothing goes past a draw.

And if someone says it is unbeatable only because it has won a lot: it is not a win record. After every burst the app searches *every* game still playable against it, in both roles, following every square you could pick and every coin flip it could make — about 15,000 complete games in some 20 milliseconds — and finds no line where it loses. That is a proof, not a sample.

WHAT IS ACTUALLY RUNNING

Tabular **afterstate value learning**: one number per board position meaning "how did this turn out for whoever just moved". Update is `+1` for a win, `0` for a draw, otherwise the best thing the opponent can do to you, negated and discounted. Learning rate **0.8**, discount **0.95**, ε from **1.00** to **0.25** over 14,000 games, **60%** self-play and **40%** versus random play in both roles, **20%** of games started from a dealt position so rare endgames get practised. No minimax, no heuristics, no opening book anywhere in the agent. Sources and the full test suite sit beside this file.

NOTES FOR THE PRESENTER

- **Short of time?** Press train twice in a row and skip Era 2. It always lands by the fifth press.
- **A student beats Era 3?** They cannot beat a *verified* era, but they can beat one before the banner. That is real and it is a gift — say so, then train again.
- **500 and 2,000** exist to show the wobble in slow motion; they need many more presses. Keep 5,000 for a timed session.
- The maths for 5,000 games finishes in well under a tenth of a second. The montage is stretched to about two and a half seconds so there is something to watch — the app admits this itself, and so should you.

SECOND ACT: NINE BOARDS AT ONCE (5 MINUTES)

The toggle at the top switches to the nine-board game: a mark per turn on *any* unfinished board, finished boards lock, first to five boards takes the match. Run this after the banner, when the room has just watched something become perfect. The point is what happens to that method when the world gets one step bigger.

0:00 **Switch to "Nine at once" and play it.**
It is beatable again — sometimes badly. Nothing was taken away from it. Ask the room why.

> "*Same opponent. Same table. It just got moved somewhere slightly bigger.*"

1:30 **Turn on "Show its brain" and read the last line.**
It reports what fraction of the squares it is weighing up sit on a board picture it has never seen — often most of them. On one board the marks tell you whose turn it is; here you can play twice in the same board while your opponent is busy elsewhere, so about half the pictures you meet *cannot happen* in ordinary tic-tac-toe. Only 14% of what this game needs came across.

2:30 **Train it, then read the first card.**
It counts, by playing, how often any position comes round twice. On one board about ten times. On nine boards about once.

> "*Learning from experience needs the experience to repeat. Here it never does. So why does this work at all?*"

Because it looks at **one board at a time** and adds up what it finds — and a person chose that shortcut. Finding the shortcut yourself is what a neural network is for.

4:00 **Point at the missing banner.**
Where Act I printed a proof, this mode prints a refusal: nobody can check every game here, so nobody can promise. Then show the score against the hand-written opponent — take any win, block any loss, else prefer the middle, the way a person would explain the game. It stops losing to those rules within a few bursts, having never been shown them.

> "*Good, but not proven. That is where every real AI system lives.*"

Two more questions for this act. (1) The agent scores a move by how much it improves the one board it is played in, and ignores the other eight. When would that be exactly right, and when would it be wrong? (2) It explores by trying whatever it has seen least, rather than at random. Why does rolling a die stop working when the space gets big?